SAT Resetmaxxing for Fun and Profit

(Final Title TBD)

Martin S. Fixman[1], Fredrik Rømming[2], and Sean B. Holden[3]

University of Cambridge,
United Kingdom
[1]msf43@cl.cam.ac.uk [2]fr409@cl.cam.ac.uk [3]sbh11@cl.cam.ac.uk

Abstract

We improve SATCoP performance by resetting the connection tableau often. This causes more and more varied SAT statements to be added to the SAT prover, which causes it to saturate faster. The resulting prover solves problems faster and in fewer steps than SATCoP and the current state-of-the-art tableau provers.

Contents

1 Introduction **1**

2 Preliminaries **1**
2.1 Tableau Solving Methods . . . 1
2.2 SATCoP and Similar Methods . . . 2

3 Research Vehicle **2**
3.1 ResetSATCoP Specification . . . 2
3.2 Other things I should add . . . 3
3.3 Python Implementation . . . 3
3.4 TPTP Benchmarking . . . 3

4 Experimental Evaluation **4**
4.1 Experiment Design . . . 4
4.2 Results . . . 4
4.2.1 Individual Results . . . 4
4.2.2 Comparative Results . . . 5
4.2.3 Soundness Verification: Problem Satisfiability . . . 6

5 Future Work **7**

[SATResetCoP or ResetSATCoP? Be consistent.]

1 Introduction

[Write an introduction that enhances and complements the abstract.]

2 Preliminaries

2.1 Tableau Solving Methods

Minimalist tableau-style provers based on Prolog have been used with great success since the release of leanTAP in 1995[1], and have consistently achieved high rankings in solving FOF and CNF problems in the Thousands of Problems for Theorem Provers (TPTP) library[12].

Rather than working like saturation provers such as Vampire[10], which repeatedly derive new clauses from the current clause set until they derive a contradiction or reach saturation,

tableau-based provers attempt to create a Connection Tableau with ground terms. The prover builds a tree, where each node corresponds to a particular clause of the problem in conjunctive normal form, using one of the following four steps.

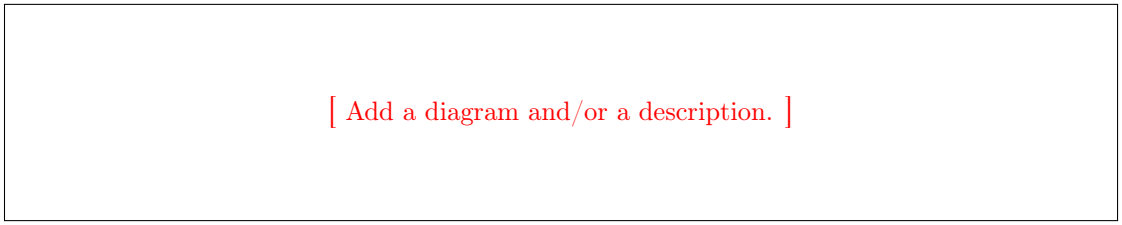


Figure 1: Description of the operations of Tableau calculus[3].

leanCoP improves on earlier connection-tableau provers by combining a very small Prolog implementation with aggressive restricted backtracking[7, 6]. Successful Reduction and Extension steps commit with a Prolog cut (!), pruning alternative continuations and often reducing search substantially. To maximise the variety of substitutions in the tableau the maximum height of the tree is constrained, and each subsequent Start increases this height. A similar technique is later used in SATResetCoP, which is presented in this paper.

meanCoP revisits this design by separating the effects of Reduction cuts and Extension cuts, and by replacing leanCoP's particularly strong pruning with more selective exclusive cuts[3]. In particular, meanCoP commits only when a successful proof step rules out sibling alternatives at the current choice point, and otherwise allows ordinary backtracking to continue[3]. This keeps much of the practical benefit of cut-based control while avoiding some of the unnecessary incompleteness and over-commitment of the original leanCoP strategy[3].

The leanCoP line has also been extended in several other directions. Functional and machine-learning-guided connection provers refine proof-step ordering and clause selection[4], rlCoP and plCoP add Monte Carlo Tree Search and reinforcement-learning-based control to leanCoP-style proof search[5, 14], and recent work on proof syntax and certification makes connection proofs easier to exchange and verify[8].

2.2 SATCoP and Similar Methods

Rawson and Reger integrate a SAT prover into a clausal tableau system based on leanCoP[9]. Grounded clauses generated in Tableau extensions are added to a SAT solver. The SAT solver resulting in an unsatisfiable set of ground clauses is sufficient to prove that the entire problem is unsatisfiable.
[Fill more information here.]

3 Research Vehicle

3.1 ResetSATCoP Specification

We propose implementing a new tableau prover, based on SATCoP, but whose tableau prover attempts to saturate the SAT prover rather than to close the tableau.

Successive experimentation resulted in a simple hypothesis: aggressively resetting the tree when the SAT solver stops growing results in a larger quantity and variety of SAT statements added to the solver, which results in reaching a SAT unsatisfiability proof faster.

ResetSATCoP[1] uses the same substitution and SAT prover unsatisfiability technique as SATCoP. However, instead of following leanCoP's tableau guidance strategy, it prioritises resetting the tableau often. [I think some pseudocode would be better than this list.]

1. Choose the next action as in SATSolveLiteral. [Elaborate].
2. Cut (!) reductions and extensions, as in LeanCoP.
3. If the next action is a Backtrack, then Reset the entire tree from scratch.
   (a) If new SAT clauses were added in this tree, keep its same maximum height.
   (b) Otherwise, attempt to make a deeper tree and increase its maximum height by 1.

Just like SATCoP, this approach is sound. While it is less complete than SATCoP and other LeanCoP variants, the speed at which it proves unsatisfiability of first-order problems can result in a larger amount of problems solved in a bounded amount of time or steps. It might be useful to incorporate ResetSATCoP into the initial steps of a more complete schedule.

3.2 Other things I should add

[Explain how I choose statements.] [Randomisation of statements.]

3.3 Python Implementation

As a testbed for out experiments we use a Python implementation of leanCoP, PyCoP. [PyCoP is as of yet unreleased. Is there anything I can cite here?] This program implements classical tableau calculus and supports leanCoP-style CUTs, different maximum heights, and adding a SAT solver to implement a SATCoP-style prover.

Most importantly, the program allows us to implement different strategies to guide the tableau proof. This makes implementing the guidance of ResetSATCoP simple and its code easy to read and evaluate.

The source code for this implementation is present in `https://github.com/mfixman/resetsatcop`, and is released under the GNU Lesser General Public Licence[2].

3.4 TPTP Benchmarking

Along with out implementation of ResetSATCoP, PyCoP allows us to reimplement leanCoP and SATCoP in its own backend. This provides a fairer baseline for comparison.

We run ResetSATCoP along with our versions of these two connection tableau strategies, along with a strategy that chooses the first possible action as control, on the 8344 CNF problems and 9091 FOF problems of all 50 categories [Categories?] containing at least one of these problems from the Thousands of Problems for Theorem Provers (TPTP) library[11].

The library provides a varied set of problems that's a standard way to prove first-order provers. It also provides known satisfiability for these problems, which is a useful way to verify soundness.

[1] [This is `SATResetAtStaleBacktrack` in the code. I plan to rename this strategy to `ResetSATCoP`.]

4 Experimental Evaluation

4.1 Experiment Design

In order to equalise comparisons in areas such as problem parsing, we use the same base Python system that parses problems using classical calculus where we implement five strategies, with properties in Table 1.

Model	Explanation	Cut	Unsat proof	Tableau Guidance
FirstAction	Simple control policy		Tableau closure	First forward action
LeanCoP	Based on leanCoP[6].	✓	Tableau closure	First forward action
SATCoP	Initial implementation of SATCoP[9].	✓	UNSAT	Random forward action
SATCoPEnhanced	Inspired by §9.1 of Rawson and Reger[9].	✓	UNSAT	SAT-Model Selection
SATResetCoP	Elaborated in Section 3.1.	✓	UNSAT	SAT-Model Selection, Reset on Backtrack

Table 1: The four control models and the test model used in this experiment.

The TPTP library contains 8344 CNF problems and 9091 FOF problems separated in 50 categories[11]. Each problem/model pair is run for a maximum of 120 seconds and 1 million steps, on a single core of the University of Cambridge CSD3 Cluster with [add CPU information] and 3370MiB RAM per core[13]
[Am I supposed to properly cite / acknowledge the use of the Cambridge cluster?]

4.2 Results

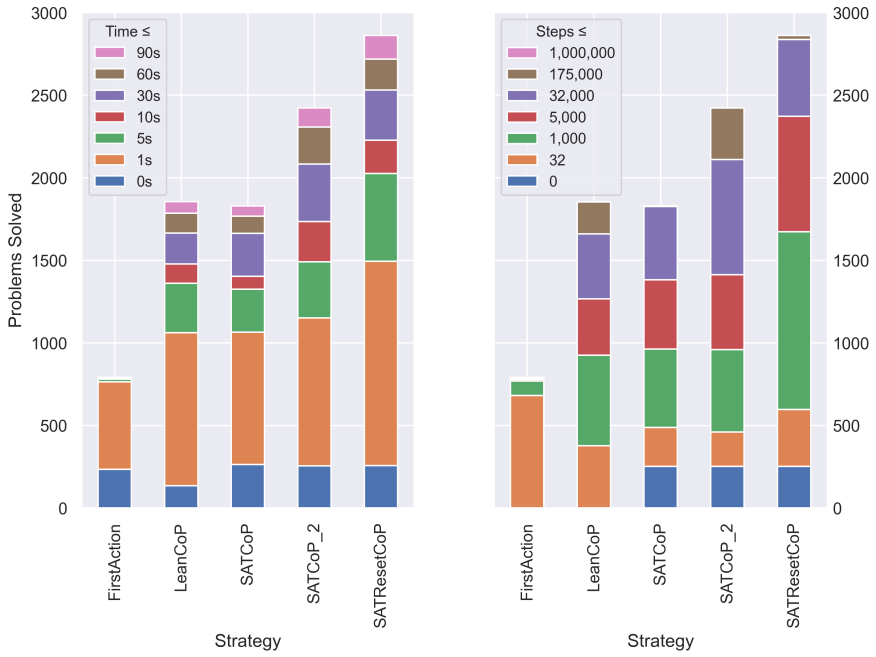
4.2.1 Individual Results

SATResetCoP considerably improves the amount of problems solved by both SATCoP models under the 120 second time limit, as shown in Table 2. This is mostly caused by the SAT solved reaching unsatisfiability in fewer time and steps than in previous models, as seen in Figures 2a and 2b.

Model	CNF	FOF	Total
FirstAction	205	587	792
LeanCopCut	701	1155	1856
SATCoP	787	1042	1829
SATCoPEnhanced	1027	1396	2423
SATResetCoP	1227	1634	2861

Table 2: Amount of CNF and FOF problems solved within the time constraint by each model.

The 1 000 000 step limit is rarely reached, and most unsolved problems hit the 120 second limit instead.



(a) Problems solved, stacked by time in seconds.

(b) Problem solved, stacked by amount of steps. SAT models can solve a problem in 0 steps if the initial grounding produces is SAT-unsatisfiable.

Figure 2: Results of running the models on all problems solved in a 120 second and 1 million step limit.

[Maximum solved time is slightly less than 90s. Is this on purpose? Probably; check the actual experiment.]

[Time comparison / violinplot of different TPTP problem types for SATResetCoP]

Figure 3: Time taken by SATResetCoP to solve each problem within each TPTP problem type. We can probably find some conclusion in here.

4.2.2 Comparative Results

The SAT guidance in SATResetCoP causes it to solve several problems that are unsolved by either models which do not have the same SAT saturation prover, and even by SATCoP models

Model	CNF	FOF
FirstAction	205	587
LeanCoP	502	693
SATCoP$_2$	188	189
SATCoP	184	129
SATResetCoP	213	228

Table 3: Problems solved by any model, not solved by models prior to them. [What's the point of separating CNF and FOF here? Maybe finding interesting TPTP types would be better.]

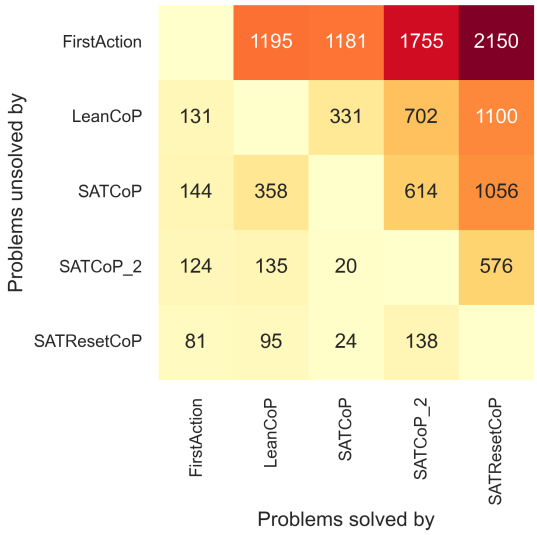


Figure 4: Pairwise comparison of model results: the cell in (column, row) shows the amount of problems solved by the model on column, but not the model in row.

that do not have similar efficient guidance. A comparative result can be seen in Table 3.

ResetSATCoP is specially strong in problems which can be solved by small unsat cores. However, even more than SATCoP, it's worse than previous models on problems which require a large amount of substitutions to produce the atoms of an unsatisfiable core. Figure 4 shows the amount of problems solved one of the models that's not solved by another one.
[There is a lot of interesting data on size of unsat cores and TPTP categories. Investigate further.]

The size of the improvement between SATResetCoP and the next best model is notable.
[Add more analysis here.]

4.2.3 Soundness Verification: Problem Satisfiability

The problems in the TPTP library have a helpful specification of the satisfiability of different types of problems[11]. This is useful as a sanity-check: a sound solver should never "prove" unsatisfiability on a satisfiable problem, and their authors should be wary when proving unsatisfiability on an open or unknown problem. Note that this is not a way to prove soundness

	FOF		CNF	
Declared status	Unsolved	Solved	Unsolved	Solved
Satisfiable	350	0	1094	0
CounterSatisfiable	732	0	0	0
Unknown	94	0	745	0
Open	4	0	28	0
Unsatisfiable	100	72	4496	1433
ContradictoryAxioms	204	39	0	0
Theorem	4426	1823	0	0

Table 4: Amount of problems solved and unsolved by ResetSATCoP, grouped by TPTP declared status. No satisfiable or countersatisfiable problem was incorrectly proven to be unsatisfiable.

of the solver, but to verify that it's not necessarily unsound.

We make a second run of SATResetCoP with a longer 10-minute limit, whose results can be seen in Table 4. These results do not show any negative results.

5 Future Work

References

[1] Bernhard Beckert and Joachim Posegga. leanTAP: Lean tableau-based deduction. 15(3):339–358, 1995.

[2] Free Software Foundation. GNU lesser general public license, version 3, 2007.

[3] Michael Färber. A Curiously Effective Backtracking Strategy for Connection Tableaux, 2024.

[4] Michael Färber, Cezary Kaliszyk, and Josef Urban. Machine Learning Guidance for Connection Tableaux. 65(2):287–320, 2021.

[5] Cezary Kaliszyk, Josef Urban, Henryk Michalewski, and Miroslav Olšák. Reinforcement learning of theorem proving. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31, pages 8836–8847. Curran Associates, Inc., 2018.

[6] Jens Otten. Restricting backtracking in connection calculi. 23(2–3):159–182, 2010.

[7] Jens Otten and Wolfgang Bibel. leanCoP: Lean connection-based theorem proving. 36(1–2):139–161, 2003.

[8] Jens Otten and Sean Holden. A Syntax for Connection Proofs, 2023.

[9] Michael Rawson and Giles Reger. Eliminating Models During Model Elimination. In Anupam Das and Sara Negri, editors, *Automated Reasoning with Analytic Tableaux and Related Methods*, volume 12842, pages 250–265. Springer International Publishing.

[10] Alexandre Riazanov and Andrei Voronkov. The design and implementation of VAMPIRE. 15(2–3):91–110, 2002.

[11] G. Sutcliffe. Stepping Stones in the TPTP World. In C. Benzmüller, M. Heule, and R. Schmidt, editors, *Proceedings of the 12th International Joint Conference on Automated Reasoning*, number 14739 in Lecture Notes in Artificial Intelligence, pages 30–50, 2024.

[12] Geoff Sutcliffe. The 12th IJCAR automated theorem proving system competition—CASC-J12. 38(1):3–20, 2025.

[13] University of Cambridge Research Computing Services. Ice Lake Nodes. https://docs.hpc.cam.ac.uk/hpc/user-guide/icelake.html, 2026. Accessed 13 May 2026.

[14] Zsolt Zombori, Josef Urban, and Chad E. Brown. Prolog Technology Reinforcement Learning Prover: (System Description). In Nicolas Peltier and Viorica Sofronie-Stokkermans, editors, *Automated Reasoning*, volume 12167, pages 489–507. Springer International Publishing, 2020.